

ゼミナールA（中島研） 自主研究について

自主研究では、深層学習（ニューラルネットワーク）を利用した識別問題を題材に、ニューラルネットワークによる研究やそのための実装で基本となる様々な要素を体感してもらいます。この自主研究で利用するデータセットは、CIFAR-10と呼ばれるもので、32x32の小さな画像を10クラスのいずれかに分類する識別問題のデータセットです。各クラスには6000枚のデータがあり、学習用に5000枚、評価用に1000枚が使われます。

<https://www.cs.toronto.edu/~kriz/cifar.html>

ニューラルネットワークに関連する実装は一から全部自力でやると大変なので、便利なフレームワークが多く公開されています。一部を列挙します。

- PyTorch <https://pytorch.org/>
- TensorFlow <https://www.tensorflow.org/>
- JAX <https://github.com/google/jax>

これらのフレームワークはいずれも、ニューラルネットワークでよく使われる要素（畳み込み層、全結合層、活性化関数、などなど）を用意しており、それらを組み合わせるだけでいろいろなニューラルネットワークを実現できます。さらに、誤差逆伝播法によって自動的に微分も計算してくれます。上記の中でもPytorchとTensorFlowがよく使われています。JAXは深層学習用のフレームワークとは少し違いますが、最近良く使われるようになっていきます。自主研究では、特に研究者に人気のPyTorchを使っていきます。

前半の輪講では、深層学習の基本的な部分を学びました。以降は、実際にニューラルネットワークのモデルを学習して、輪講で学んだ様々な要素が実際のモデルの性能にどのように影響するかを研究してもらいます。ニューラルネットワークの実装では、データの読み込み・準備、学習ループ、モデルの定義が必要になります。フレームワークを利用しても、データの読み込み・準備や学習ループは大変なので、ここではGoogle Colaboratory上で一通り動くコードを用意しています（基本的には、PyTorchのウェブサイトにあるチュートリアルとほぼ同じものです）。

Google Colaboratoryは、Pythonをウェブブラウザ上で気軽にコーディングできるようにしたIPython Notebookと呼ばれるものを、Googleがサービスとして提供しているものです。コードを入力する部分はセルと呼ばれています。Colaboratoryに限らず、IPython Notebookではセルごとに処理を実行しますが、その結果はすべて共有されています。つまり、一つのセルで定義した変数や関数などを他のセルでも使うことができます。時間のかかる処理をあるセルで実行しておいて、その結果を可視化したり後処理したりといった部分を別のセルにすることで、時間のかかる処理を1回で済ませることができるというメリットがありますが、一方であるセルで定義した変数や関数、クラスを別のセルで上書きしてしまう可能性もあるので注意してください。

画像の表示やグラフの描画などの可視化にはmatplotlibを使います。一次情報は

<https://matplotlib.org/>

ですが、ネット上に日本語の記事を多く見つけることができますので、そちらを参考にしてください。この自主研究では、matplotlib.pyplotのimshow（画像を表示）、plot（折れ線グラフなど）、bar（棒グラフ）を使えば十分かもしれませんが、他にも多くの機能があります。また、グラフの描画はエクセルなどでも大丈夫です。

深層学習は多くの人達が試して日本語でネット上に記事を書いてくれています。上記の公式ウェブサイトにあるドキュメントなどに加えて、これらの日本語での情報を活用して自主研究に取り組んでください。

GPUが使えるか確認

```
import torch
torch.cuda.is_available()
```

1行目は、**torch**というライブラリを使うための準備。実際の名前は**PyTorch**。

2行目は**CUDA**（GPUをグラフィックス以外で利用するためのもの）が利用できるかの確認。**Google Colaboratory**では**NVIDIA**のGPUを使うことができます。もし上記セルの実行結果が**False**の場合は、ランタイムを変更してください。

モデルの定義

```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Model(nn.Module):
    def __init__(self):
        super(Model, self).__init__()
        self.conv1_1 = nn.Conv2d( 3, 64, 3, padding=1)
        self.bn1_1 = nn.BatchNorm2d(64)
        self.conv1_2 = nn.Conv2d(64, 64, 3, padding=1)
        self.bn1_2 = nn.BatchNorm2d(64)
        self.conv1_3 = nn.Conv2d(64, 64, 3, padding=1)
        self.bn1_3 = nn.BatchNorm2d(64)
        self.conv1_4 = nn.Conv2d(64, 64, 3, padding=1, stride=2)
        self.bn1_4 = nn.BatchNorm2d(64)

        self.conv2_1 = nn.Conv2d(64, 128, 3, padding=1)
        self.bn2_1 = nn.BatchNorm2d(128)
        self.conv2_2 = nn.Conv2d(128, 128, 3, padding=1)
        self.bn2_2 = nn.BatchNorm2d(128)
        self.conv2_3 = nn.Conv2d(128, 128, 3, padding=1)
        self.bn2_3 = nn.BatchNorm2d(128)
        self.conv2_4 = nn.Conv2d(128, 128, 3, padding=1, stride=2)
        self.bn2_4 = nn.BatchNorm2d(128)

        self.conv3_1 = nn.Conv2d(128, 256, 3, padding=1)
        self.bn3_1 = nn.BatchNorm2d(256)
        self.conv3_2 = nn.Conv2d(256, 256, 3, padding=1)
        self.bn3_2 = nn.BatchNorm2d(256)
        self.conv3_3 = nn.Conv2d(256, 256, 3, padding=1)
        self.bn3_3 = nn.BatchNorm2d(256)
        self.conv3_4 = nn.Conv2d(256, 256, 3, padding=1, stride=2)
        self.bn3_4 = nn.BatchNorm2d(256)

        self.fc4 = nn.Linear(256, 256)
        self.fc5 = nn.Linear(256, 256)
        self.fc6 = nn.Linear(256, 10)
```

```
def forward(self, x):
    x = F.relu(self.bn1_1(self.conv1_1(x)))
    x = F.relu(self.bn1_2(self.conv1_2(x)))
    x = F.relu(self.bn1_3(self.conv1_3(x)))
    x = F.relu(self.bn1_4(self.conv1_4(x)))

    x = F.relu(self.bn2_1(self.conv2_1(x)))
    x = F.relu(self.bn2_2(self.conv2_2(x)))
    x = F.relu(self.bn2_3(self.conv2_3(x)))
    x = F.relu(self.bn2_4(self.conv2_4(x)))

    x = F.relu(self.bn3_1(self.conv3_1(x)))
    x = F.relu(self.bn3_2(self.conv3_2(x)))
    x = F.relu(self.bn3_3(self.conv3_3(x)))
    x = F.relu(self.bn3_4(self.conv3_4(x)))

    x = torch.mean(x.view(x.size(0), x.size(1), -1), dim=2)
    x = F.dropout(x)
    x = F.relu(self.fc4(x))
    x = F.dropout(x)
    x = F.relu(self.fc5(x))
    return self.fc6(x)
```

Pythonのクラス定義（nn.Moduleを継承）

コンストラクタ

nn.Moduleのコンストラクタを実行

必要な変数の定義。学習が必要な（＝パラメータを持つ）層を作っておく。

今回は学習が必要な層は

- ・2次元畳み込み層 nn.Conv2d

<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

- ・バッチ正規化層 nn.BatchNorm2d

<https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html#torch.nn.BatchNorm2d>

- ・全結合層 nn.Linear

<https://pytorch.org/docs/stable/generated/torch.nn.Linear.html#torch.nn.Linear>

畳み込み層のコンストラクタ引数は

- ・入力チャンネル数
- ・出力チャンネル数
- ・カーネルの大きさ

padding=1は入力画像の外側に1画素分余白を追加。カーネルの大きさが3なので、paddingが0の場合は画像の各辺で1画素分小さくなるが、paddingを1にすると入力画像の各辺が1画素ずつ大きくなるので、入力と出力のサイズが一致する。

畳み込みを計算するときに、普通は1画素ずつカーネルをずらして行くところを、stride=2とすると2画素ずつずらす。結果として、出力画像は半分のサイズに。

入力画像x（実際はバッチ）に対して予測を実行する関数。Feedforward Networkのforward。

基本的には、入力に対して、

畳み込み層→バッチ正規化層→活性化関数（ReLU）を順番に適用するのみ。

ここでは（変数の定義のところも）、畳み込み層4つずつでひとまとまりにした。それぞれのまとまりで、最初の畳み込み層でチャンネル数を増やし、最後の畳み込み層で空間方向のサイズ（画像の幅と高さ）を半分に。

これによって、同じカーネルサイズ（ここでは3）でも、より広い範囲を見ることに。最後のまとまりでは、元の画像サイズで見れば12x12のカーネルに相当。元の画像サイズが32x32なので、だいぶ大きい。

まず、画像（幅と高さをもつデータ）をベクトルに。Dropoutはベクトルの要素の100%をランダムに0にする（デフォルトでp=0.5）。

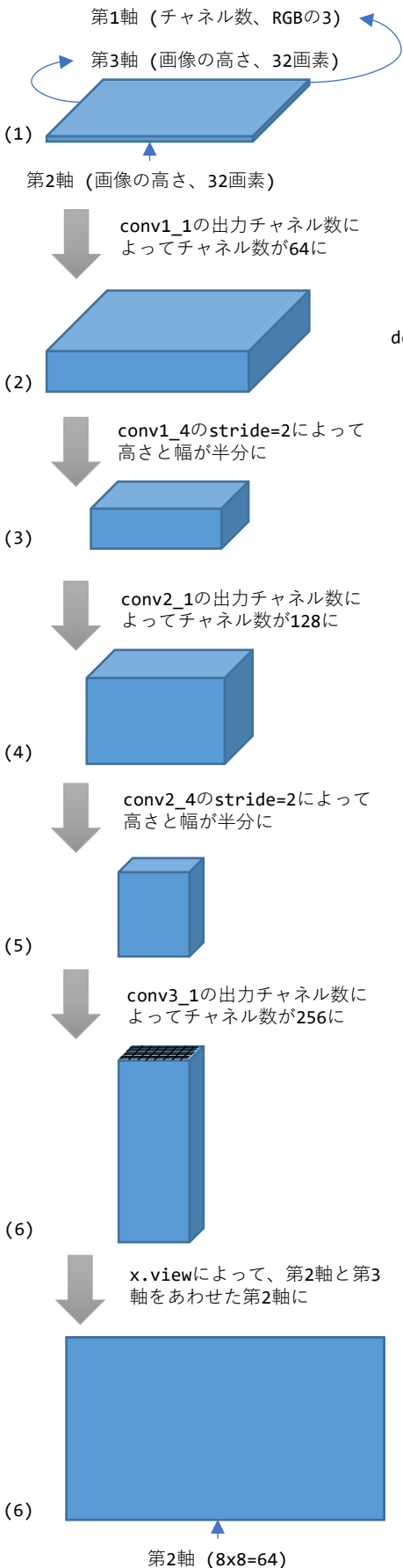
その後は、全結合層とReLU（とDropout）を繰り返して、最後は10次元のベクトルをリターン。

元の画像サイズが32x32なので、Forward()の引数として与えられた時点で、xは(バッチサイズ, チャンネル数3, 高さ32, 幅32)のテンソル。torch.mean()の時点でstride=2の畳み込みが2回適用されており、チャンネル数は256なので、xは(バッチサイズ, チャンネル数256, 高さ8, 幅8)のテンソル。

多くのCNNでは、畳み込み層の後に全結合層を利用するが、全結合層はベクトルしか入力できない（入力の形が(バッチサイズ, ベクトルサイズ)になる必要がある）ので、xをベクトルにする必要がある。ここでは、空間方向に各チャンネルの平均値を計算。x.viewはテンソルの形を変換する（したように見せる）で、ここでは3階のテンソルにしており、最初の2つは元のxのサイズをそのまま、最後の-1は、残りのサイズをまとめて1軸にする。つまり、(バッチサイズ, チャンネル数, 高さ, 幅)の4階のテンソルを(バッチサイズ, チャンネル数, 高さx幅)の3階のテンソルに。torch.mean()は、dimで指定された軸で平均を計算する。ここではdim=2なので、第2軸（高さx幅の部分）を潰すように平均を計算。

モデルの定義（forwardの図解）

畳み込み層でどのようにxが変化していくかを図解します。本来は4階のテンソルで、その形は(バッチサイズ, チャンネル数, 高さ, 幅)ですが、4次元の図が書けないので、バッチに相当する軸（第0軸）は省略します。実際には、同じものがバッチサイズだけあると考えてください。バッチサイズは、ここではとりあえず512としておきます。



```
def forward(self, x):  
    # (1) x.shape = (512, 3, 32, 32)  
    x = F.relu(self.bn1_1(self.conv1_1(x))) # (2) x.shape = (512, 64, 32, 32)  
    x = F.relu(self.bn1_2(self.conv1_2(x)))  
    x = F.relu(self.bn1_3(self.conv1_3(x)))  
    x = F.relu(self.bn1_4(self.conv1_4(x))) # (3) x.shape = (512, 64, 16, 16)  
  
    x = F.relu(self.bn2_1(self.conv2_1(x))) # (4) x.shape = (512, 128, 16, 16)  
    x = F.relu(self.bn2_2(self.conv2_2(x)))  
    x = F.relu(self.bn2_3(self.conv2_3(x)))  
    x = F.relu(self.bn2_4(self.conv2_4(x))) # (4) x.shape = (512, 128, 8, 8)  
  
    x = F.relu(self.bn3_1(self.conv3_1(x))) # (5) x.shape = (512, 256, 8, 8)  
    x = F.relu(self.bn3_2(self.conv3_2(x)))  
    x = F.relu(self.bn3_3(self.conv3_3(x)))  
    x = F.relu(self.bn3_4(self.conv3_4(x)))  
  
    x = torch.mean(  
        x.view(x.size(0), x.size(1), -1), # (6) x.shape = (512, 256, 64)  
        dim=2  
    )  
    # (7) x.shape = (512, 256)  
    x = F.dropout(x)  
    x = F.relu(self.fc4(x))  
    x = F.dropout(x)  
    x = F.relu(self.fc5(x))  
    return self.fc6(x) # (8) x.shape = (512, 10)
```

モデルの学習

Pythonノートブック（Google Colaboratory）と順番は前後しますが、実際に学習を行うために必要な準備段階のコードを見てみます。ちなみに、CIFER1-10のクラスはノートブックの通り、airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truckで順番に0から9までのラベルが割り振られています。

Colaboratory上で実行する際には、カレントディレクトリ上にmodelとlogsというディレクトリが必要です。なければ作っておいてください。（ファイルは、Colaboratoryの画面の左端にある  アイコンをクリックするとブラウザでできます）

<pre>import torch import torchvision import torchvision.transforms as transforms import torch.optim as optim import torch.nn as nn import torch.nn.functional as F from torch.utils.tensorboard import SummaryWriter import datetime</pre>	必要なライブラリを準備。 Pythonには、ライブラリに短い名前を付ける機能があります（<code>import xxx as yyy</code>のxxxが元のライブラリ名、yyyが短い名前）。長い名前をいちいち書かなくて済みます。 また、ライブラリから特定の関数やクラスなどだけを使えるようにする機能もあります。from xxx import yyyがその部分で、xxxのライブラリの中のyyyの関数やクラスなどが使えるようになります。
<pre>transform_train = transforms.Compose([transforms.RandomCrop(32, padding=4), transforms.RandomHorizontalFlip(), transforms.ToTensor(), transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),]) transform_test = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),])</pre>	データセットからデータを取り出すときに自動的に適用される操作を指定。学習セットでは、RandomCrop, RandomHorizontalFlip, ToTensor, Normalize、評価セットではToTensorとNormalizeのみ。 - RandomCrop: 元の画像から第1引数のサイズをランダムに切り出す。はみ出したところはpaddingで指定された方法で埋める。 https://pytorch.org/vision/main/generated/torchvision.transforms.RandomCrop.html - RandomHorizontalFlip: 元の画像を100%の確率で水平方向に反転。Pのデフォルト値は0.5 https://pytorch.org/vision/stable/generated/torchvision.transforms.RandomHorizontalFlip.html - ToTensor: 元のデータをPyTorchのTensorクラスに変換 - Normalize: 画像を正規化。第1引数が画素値の平均、第2引数が画素値の標準偏差。RGBそれぞれに適用。入力画像のある画素のRGBいずれかをx、出力画像の対応する値をy、平均m、標準偏差sとすると、$y = (x - m) / s$ https://pytorch.org/vision/main/generated/torchvision.transforms.Normalize.html
<pre>trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform_train) trainloader = torch.utils.data.DataLoader(trainset, batch_size=512, shuffle=True, num_workers=2)</pre>	ライブラリtorchvisionの機能でデータセットを自動ダウンロード。ダウンロードしたデータはカレントディレクトリ内のdataに保存。 transform=transform_trainによって上のブロックの操作をするように指定。 DataLoaderは、データセットから自動的にバッチを作るもの。学習のときは、データセットのデータを繰り返し使うが、このとき同じ順番でデータが出てくると学習に偏りができるため、毎回違う順番でデータが出てくるようにする（shuffle=True）。ここでは、trainsetを指定して学習セットからバッチを作る。バッチサイズは512。
<pre>testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform_test) testloader = torch.utils.data.DataLoader(testset, batch_size=512, shuffle=False, num_workers=2)</pre>	同上。ただし、評価セットではデータが同じ順場に出てきてもいいので、shuffle=False。

2つ目のブロックで、RandomCropとRandomHirozontalFlipはデータ拡張（Data Augmentation）の一種。データが増えるように見える。（参考: <https://qiita.com/shoheihoh/items/137141864394c3e4866b>）RandomCropはここでは32x32の画像から32x32の領域を切り出そうとしており、同じサイズなので少し平行移動したものが出てくる。はみ出した部分は黒で埋められる。

モデルの学習の続き

<code>model = Model()</code>	前のセルで定義したモデルのインスタンスを作成
<code>device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")</code> <code>model.to(device)</code>	GPUが使える場合は、計算用デバイスをGPUに設定。使えなければCPUに。続いてモデルのインスタンスを計算用デバイスのメモリに転送（CPUの場合は特に何もしない）。
<code>criterion = nn.CrossEntropyLoss()</code>	学習に使うロスを変数 <code>criterion</code> に入れておく。ここではCross Entropy Loss。
<code>optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9, weight_decay=0.0001)</code>	学習に使う最適化アルゴリズムを変数 <code>optimizer</code> に入れておく。ここでは、確率的勾配降下法（Stochastic Gradient Descent; SGD）。最適化の対象となるパラメータを登録する必要があるが、 <code>model.parameters()</code> がいろいろにしてくれる。他の引数は下で解説。
<code>timestamp = datetime.datetime.now().strftime("%Y%m%d-%H%M%S")</code>	学習経過を記録するログファイルの名前に付けるための現在時刻。
<code>model_dir = './models'</code> <code>log_dir = './logs/cifar10_' + timestamp</code> <code>writer = SummaryWriter(log_dir=log_dir)</code>	学習したモデルの保存場所と、ログファイルの場所。SummaryWriterを利用してここで指定したログファイルに学習の経過を記録しておく。
<code>train(model, optimizer, criterion, trainloader, testloader, writer, val=1, model_path=model_dir)</code>	Colaboratory上では上のセルで定義される学習ループのための関数を、必要な引数とともに呼び出す。Train()は次ページで解説。

4つ目のブロックでは最適化アルゴリズムを設定している。これには、モデルのパラメータを登録している必要がある。モデルはCPUのメモリかGPUのメモリのいずれかにある（`device`がどうなっているかによる）ため、必ず`model.to(device)`などによって実際に使う計算用デバイスのメモリに転送した後に最適化アルゴリズムに登録する。

また、SGDには学習率（learning rate; `lr`）、モーメンタム（momentum）、Weight Decayなどの設定が必要。モーメンタム、Weight Decayは多くの場合書かれている程度の値が使われるが、学習率はバッチ数などによって調整が必要とされている。具体的には、（モーメンタムとWeight Decayがない場合）SGDによる学習規則は以下の式の通りで、学習率は α 。

$$\Theta \leftarrow \Theta - \alpha \frac{\partial \ell}{\partial \Theta}$$

学習率が小さいと、学習するべきパラメータ Θ の更新幅が小さくなり、学習が終わらない。学習率が大さいと、ロスの極小値の周りを振動する可能性があり、極小値まで到達しない。学習の様子を見ながら決める必要がある。

学習ループの定義

学習ループでは、データセットを全体を何回走査するか（1回分を1エポックと呼ぶ）を決めて、その回数ループを回します。各エポックでは、データローダ（インスタンスの変数名はtrainloader）を使ってデータセットから1バッチ分のデータ（画像512枚と対応するラベル）を取り出して、そのデータをモデルに入力して、出力のテンソル（サイズは(バッチサイズ, 出力次元数) = (512, 10)）を受けます。この出力とラベルからロス（ここでは、Cross Entropy Loss）を計算して、このロスが小さくなるように誤差逆伝播法で最適化します。

ニューラルネットワークの学習では、ロスがどのように変化しているかが重要になります。いざ学習をはじめてみても、全くロスが下がらないことがあったり、数エポックで小さい値に収束することもあります。このページのコードの後半は、ロスの値を記録するためのコードです。

<pre>def train(model, optimizer, criterion, trainloader, testloader, writer, val=10, model_path=None): counter = 0 for epoch in range(20):</pre>	とりあえず20エポックとしている。 本番用はもっと回したほうが良いかも 勾配計算が必要なことをmodelに伝える
<pre> model.train() running_loss = 0.0 running_corr = 0 running_num = 0</pre>	以降でバッチごとのロス（と、データを何個入力してそのうち何個が正解したか）を計算する。その値をしばらく積算。 ロスの記録を残すためのもの。
<pre> for i, data in enumerate(trainloader, 0):</pre>	dataは(バッチサイズ分の画像, 対応するラベル)のtuple
<pre> inputs, labels = data inputs = inputs.to(device) labels = labels.to(device)</pre>	Tupleからそれぞれを取り出して、計算デバイス（ここではGPU）に転送。
<pre> optimizer.zero_grad()</pre>	前に計算した勾配が残っているのでゼロにリセット
<pre> outputs = model(inputs) loss = criterion(outputs, labels) loss.backward() optimizer.step()</pre>	画像をモデルに入力して、識別結果(output)を受ける。 outputsに対してロスを計算して（criterion）、誤差逆伝播して（backward）、モデルのパラメータを更新（optimizer.step）。
<pre> running_loss += loss.item() running_corr += (torch.max(outputs.data, 1)[1] == labels).sum().item() running_num += outputs.size()[0] if counter % 10 == 9:</pre>	ロスの変化を記録。 Torch.max(output.data, 1)はバッチ内のそれぞれのデータに対する出力（outputのテンソルの第1軸、第2引数で指定）の中で最大となる値と、そのインデックスのtupleを返す。
<pre> writer.add_scalar('train/loss', running_loss/10, counter+1) writer.add_scalar('train/accuracy', running_corr/running_num, counter+1) running_loss = 0.0 running_corr = 0 running_num = 0 counter += 1</pre>	Torch.max(output.data, 1)[1]はインデックスの方。“== labels”によって、バッチ内のデータそれぞれについて、ラベルと等しければ1、違えば0のベクトルが得られる（ベクトルのサイズはバッチサイズ）。これの和（sum）を取ると、バッチ内で正解したデータ数。 .item()はTensorが0階（スカラ）のとき、その値を（勾配計算をしないようにして）返す。

ニューラルネットワークのプログラミングで難しいところは、データの型、そのデータがどこにあるのか、さらにデータのサイズを意識しておかないといけないところです。

Pythonでは変数の型を明示しないので、変数にどのような型のデータが入っているのか、自分で把握しておく必要があります。PyTorchで利用するデータの型はだいたいの場合torch.Tensorで、上記のコードもデータや計算結果が入っている変数はほとんどtorch.Tensorです。ただ、.item()で取り出したデータはPythonの浮動小数点型になります。running_lossなどは学習経過の記録のためのものなのでパラメータの更新には無関係で、勾配計算は不要ですが、torch.Tensor型をそのまま使うと、不要な勾配計算が発生します。

また、データがどの計算デバイスにあるかも意識する必要があります。今回はGPUを利用するので、.to(device)をした時点でそのデータはGPUに転送されます。printで表示したい場合や、ログに記録したい場合はGPUのデータを直接使うことができないので、.item()などによってCPUメモリ上にデータを転送する必要があります（上のコードで.item()を使う一番の理由はこれです）。

最後に、データのサイズを意識する必要がある点について。torch.Tensorの場合（numpy.ndarrayの場合も）、データは多次元配列に保持されています。多次元配列は数学的に言えばテンソルに相当するものなので、Tensorという名前が使われています。多次元配列には、まずその多次元配列が何次元の配列か（テンソルの階数）、さらに配列の各次元（テンソルの軸）に何個要素があるか（テンソルの軸が何次元か）があります。この形（torch.Tensorやnumpy.ndarrayでは、.shapeによってこれらの情報が得られます）を常にはアックする必要があります。例えば、上のコードで、inputsは(512, 3, 32, 32)のテンソル、labelsは(512,)のテンソル（ベクトル）になるはずで、outputsは(512, 10)のテンソル（行列）で、criterionでロスを計算するとスカラになります。

学習ループの続き

```
## for epoch in range(10000): 内

    if epoch % val == val-1:
        model.eval()
        accum_loss = 0.0
        accum_corr = 0
        accum_num = 0
        with torch.no_grad():
            for i, data in enumerate(testloader, 0):
                inputs, labels = data
                inputs = inputs.to(device)
                labels = labels.to(device)

                outputs = model(inputs)
                loss = criterion(outputs, labels)

                accum_loss += loss.item()
                accum_corr += (
                    torch.max(outputs.data, 1)[1] ==
                    labels).sum().item()
                accum_num += outputs.size()[0]

            writer.add_scalar(
                'test/loss',
                accum_loss/i,
                counter)
            writer.add_scalar(
                'test/accuracy',
                accum_corr/accum_num,
                counter)
            print(
                'epoch:', epoch+1,
                'test loss:', accum_loss/i,
                ' test accuracy:', accum_corr/accum_num)

    model.train()
    if model_path is not None:
        torch.save(
            model.state_dict(),
            model_path+'/%05d.pth' % (epoch+1))
```

検証用なので、学習のための勾配計算は不要

検証用なので、学習のための勾配計算は不要

各エポックで、モデルを指定されたパスに保存

学習ループの後半は、検証用セットを利用してモデルの性能を確認するためのものです。やることは学習用セットでやったこととほとんど同じですが、学習ではないのでパラメータの更新はしません。なので勾配の計算も不要です。そのために、モデルに学習ではないことを伝え（`model.eval()`）、実際の計算のところでは `with torch.no_grad():` によって、勾配計算をしないようにしています。

注意：このコードでは、**CIFAR-10**の評価用セットを利用して検証しています。さらに、次のページでは、実際の性能評価にも同じ評価用セットを利用しています。これは、本来やってはいけないことですが、簡単のためにこのような実装にしています。

学習したモデルの確認

```
import torch
import torchvision
import torchvision.transforms as transforms
import torch.optim as optim
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.tensorboard import SummaryWriter

import numpy as np
import datetime
import matplotlib.pyplot as plt

%matplotlib inline

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        (0.4914, 0.4822, 0.4465),
        (0.2023, 0.1994, 0.2010)),
])

testset = torchvision.datasets.CIFAR10(
    root='./data', train=False,
    download=True,
    transform=transform_test)
testloader = torch.utils.data.DataLoader(
    testset,
    batch_size=1,
    shuffle=False,
    num_workers=2)

model = Model()

model_sd = torch.load('models/00020.pth')
model.load_state_dict(model_sd)

device = torch.device(
    "cuda:0" if torch.cuda.is_available()
    else "cpu")
model.to(device)

model.eval()

dataiter = iter(testloader)

cnt = 0
for x, l in dataiter:
    x = x.to(device)
    y = F.softmax(model(x), dim=1)
    y = y.to('cpu')
    print(y.detach().numpy())
    print(l.item())

    img = x.to('cpu').numpy().reshape(
        (3, 32, 32)).transpose((1,2,0))
    img = img * np.array(
        (0.2023, 0.1994, 0.2010)).reshape((1,1,-1))
    img = img + np.array(
        (0.4914, 0.4822, 0.4465)).reshape((1,1,-1))

    plt.imshow(img)
    plt.show()
    cnt += 1
    if cnt == 4:
        break
```

保存したモデルのロード。ファイルは`state_dict`と呼ばれる形でモデルを保存しており、それを実際のモデルに変換。ファイル名は検証用セットで精度が一番良いモデルのものを選ぶとよい。

画像1枚ずつモデルに入力。データ`x`の形はバッチサイズ1のバッチ`(1, 3, 32, 32)`。出力に対して`softmax`を適用。この時点で`y`はGPU上にあるが、`print`するためにCPUに転送して`numpy.ndarray`に変換。`l`は`torch.Tensor`で、値を一つだけ持つため、`.item()`が使える。

画像を1枚ずつ表示。

最初の4枚を表示したらループを抜ける。

上のコードの2つ目のブロックで、モデルの出力に対して`softmax`を適用しているが、これは予測されたラベル（モデルの出力で一番大きな値を持つインデックス）を出力するだけであれば不要。ここでは、確率として見るために適用。

自主研究について

自主研究では、ここまでのコードを利用してニューラルネットワークの何を変えたら性能がどう変わるのかを検証します。

その前の練習問題として、以下の`matplotlib`を利用した各種可視化をしてみましょう。

- モデルに入力された画像を表示する
- 前のページで、モデルの動作を確認するために、モデルの出力に`softmax`を適用した。この出力を棒グラフで表示する
- 学習用セット、検証用セット（ここでは、評価用セットと同じ）の両方について、学習曲線（ロスや精度を線グラフで表示したもの）を表示する（ロスは`running_loss`と`accum_loss`、精度は`running_corr/running_num`と`accum_corr/accum_num`）。学習中のこれらの値を記録する必要がある。このとき、学習用セットの場合は、`running_loss`が1エポック内で何度もリセットされることに注意。単純に配列に記録していくだけだと、学習用セットと評価用セットのタイミングが一致しない。

上記ができたら、実際に自主研究です。例として、以下の点についてモデルの（学習用セットと評価用セット両方での）精度に土曜な影響を与えるかを検証することが考えられます。全員で話し合って、（以下の項目に限らず）それぞれの担当を決めてください。

- 入力画像に対する前処理（データ拡張）のやめるとどうなるか、また他のデータ拡張手法ではどうなるか
- 入力画像に対する正規化（`Normalize`）は本当に必要か
- 学習率を変えるとどうなるか
- 学習時のバッチサイズについて
- `Dropout`の有無と、`Dropout`の割合`p`を変えるとどうなるか
- バッチ正規化（`BatchNormalization`）の有無
- 活性化関数による違い
- 畳み込み層の数を増減してみる、またはチャンネル数（隠れ状態の次元数）を増減してみる
- 全結合層の数を増減してみる、または隠れ状態の次元数を増減してみる

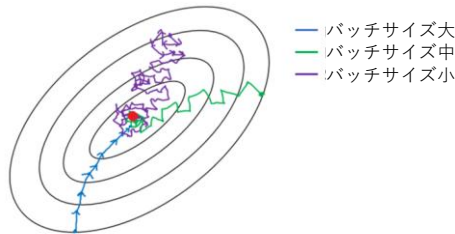
結果はグラフにまとめるなどして、発表のときに見る人にもわかりやすくしましょう。また、なぜそうなるのかを考察してみましょう。

データ拡張について

データ拡張は、少し違うものでも同じ識別結果にするためのものです。上記コードでは、上下左右に少しだけ平行移動するもの（`Random Crop`）や、水平方向に反転させるもの（`Random Horizontal Flip`）を使っており、これらによって平行移動や水平方向があっても同じ画像は同じだとモデルに学習させることができます。例えば、もし平行移動をしなければ、もしかしたら1ピクセル右にずれただけで全く違う識別結果を出力するかもしれません（実際にはそんな極端なことはあまり起こりませんが）。これらをやめたらどうなるのか、または他のデータ拡張（ノイズを付加する、一部を消す）を導入したら実際にそのような変化に対して頑健になるかを見てみると面白いかもしれません。

入力画像に対する正規化について

上のコードでは、元々`[0, 1]`の範囲の値を持つ入力画像を平均`0`、分散`1`に正規化しています。画像を入力とする場合、この処理がほとんどの場合導入されています。これは、例えば値の範囲が`RGB`のそれぞれで順に`[0, 1]`、`[0, 0.001]`、`[0, 0.001]`だとすると、`R`だけが出力に大きな影響を与えることになります。ただし、このデータでは、`RGB`のいずれも平均が`0.5`付近、分散が`0.2`付近になっているので、なくてもあまり影響はないかもしれません。実際に、正規化する場合としない場合を比べてみてどうなるか、またその理由を考えてみましょう。



学習率とバッチサイズについて

学習率は上記コードの解説に書いたとおり、1回のパラメータの更新でどれだけ値を変化させるかを決めています。これを大きくすると早く最適化が進む可能性がありますが、大きすぎるとうまく最小値に到達できず、振動したり発散したりします。小さくするとなかなか学習が進まなくなり、さらにあまり良くない極小値に捕まってしまう可能性も大きくなります。一方で、バッチサイズが小さいと、一つのデータのみからの勾配でパラメータを更新することになり、行き先がランダムに近くなります。バッチサイズが大きいと多くのデータの勾配の平均を考えることになるので、すべてのデータにとっての最適値に早く近づくように更新するようになります。ただし、多くのデータを見るということは勾配の計算に時間がかかることになりますし、動きも小さくなります。バッチサイズと学習率は同時に色々変えてみてどういうときに一番良くなるかを考える必要があります。

Dropoutについて

`Dropout`は隠れ状態の値をランダムに`0`にするものです。ランダムに一部の値を`0`にされても同じ画像に同じ出力をしてほしいので、`Dropout`を入れると隠れ状態が冗長になります（つまり、違う値が高い相関を持つ）。これによって過学習を防ぐことができます。

BatchNormalizationについて

`BatchNormalization`は、バッチ内で隠れ状態の値の範囲を平均`0`、分散`1`に正規化するものです。これは活性化関数に密接に関わってきます。というのも、正規化がないと、ある層の出力が正負で大きな値になる可能性があり、`sigmoid`や`tanh`などの活性化関数では勾配が`0`になってしまいます。正規化するとこの状況は防げますね。この有無がどのように影響を与えるのかを見てみましょう。

活性化関数の違い

活性化関数として、昔は`sigmoid`や`tanh`が使われていました（今でも使われることはあります）。ただ、`ReLU`の登場以降は畳み込みニューラルネットワークの多くは`ReLU`やその変種を利用しています。`BatchNormalization`と合わせて、活性化関数が最終的な性能にどのような影響を与えるのか調べてみましょう。